Computer Organisation

Abhijat Vichare

Flame University

Computer Organisation: A Summary

August 2023

Key Theme

Key Idea of this Course

Abstractions

Expose essential, Hide unnecessary.

Computing: Transform information to obtain new information.

Computing systems are built as levels of abstractions.

Key Theme

To build a computing system we need

- **Algorithms**: Capture the finite set of steps of processing
  - Mathematical Models of Computation: What algorithms are.
  - Complexity: How good the algorithms are, if they exist, for a given process
- **Languages**: Express algorithms to “Execution” systems
  - Various kinds of language designs to relieve programming effort
  - Procedural (C, C++, Python, Java), OOP (C++, Java, Python), Functional (Lisp, Scheme, Haskell), Logic (Prolog), Strings based (Snobol), ...
- **Execution**: Combination of Hardware + Software to execute programs
  - Hardware:
    - CPU (Transformations),
    - Memory (Primary and Secondary) and
    - Input/Output devices
  - Operating Systems and Systems Software: Processes and Files

Key Theme

To build a computing system some of the things we need are:

- Algorithms: Capture the finite set of steps of processing
  - Mathe
  - Comp
- Langu
  - Variou
  - Proce Lisp, Scher
- Execu ams
  - Hardw
    - CPU
    - Memory (Primary and Secondary) and
    - Input/Output devices
  - Operating Systems and Systems Software: Processes and Files

Key Idea

Abstraction

Hide unnecessary detail and focus on the essential.

Key Theme

To build a computing system some of the things we need are:

- Algorithms: Capture th
  - Mathematic
  - Complexity: How good t
- Languages: Express a
  - Various kinds of languag
  - Procedural (C, C++, Python, Java), OOP (C++, Java, Python), Functional (Lisp, Scheme, Haskell), Logic (Prolog), Strings based (Snobol), ...
- Execution: Combination of Hardware + Software to execute programs
  - Hardware:
    - CPU (Transformations),
    - Memory (Primary and Secondary) and
    - Input/Output devices
  - Operating Systems and Systems Software: Processes and Files

Separate courses:

- Theory of Computation
- Design and Analysis of Algorithms

Key Theme

To build a computing system some of the things we need are:

- Algorithms: Capture th
  - Mathematical Models of
  - Complexity: How good t
- Languages: Express
  - Various kinds of langua
  - Procedural (C, C++, Python, Java), OOP (C++, Java, Python), Functional (Lisp, Scheme, Haskell), Logic (Prolog), Strings based (Snobol), ...
- Execution: Combination of Hardware + Software to execute programs
  - Hardware:
    - CPU (Transformations),
    - Memory (Primary and Secondary) and
    - Input/Output devices
  - Operating Systems and Systems Software: Processes and Files

Separate courses:

- Programming using language X
- Paradigms of Programming

Key Theme

To build a computing system some of the things we need are:

- Algorithms: Capture th
  - Mathematical Models of
  - Complexity: How good t
- Languages: Express a
  - Various kinds of languag
  - Procedural (C, C++, Python, [illegible] (Lisp, Scheme, Haskell), Logic ([illegible]), Strings based (Snobol), ...
- Execution: Combination of Hardware + Software to execute programs
  - Hardware:
    - CPU (Transformations),
    - Memory (Primary and Secondary) and
    - Input/Output devices
  - Operating Systems and Systems Software: Processes and Files

Separate courses:

- Computer Architecture
- Systems Software
- Operating Systems

Key Theme

To build a computing system some of the things we need are:

- Algorithms: Capture the
  - Mathematical Models of
  - Complexity: H
- Languages: Expres
  - Various kinds of languag
  - Procedural (C, C++, Pyt isp, Scheme, Haskell), Logic
- Execution: Combinatio ms
  - Hardware:
    - CPU (Transformations),
    - Memory (Primary and Second
    - Input/Output devices
  - Operating Systems and Systems Software: Processes and Files

Abstractions:

- Theory of Computation

  Use a mathematical system, Turing Machines, to model computation.

- Design and Analysis of Algorithms

  Use a mathematical system to measure algorithm performance.

Key Theme

To build a computing system some of the things we need are:

- Algorithms: Capture the
  - Mathematical Models of
  - Complexity: How go
- Languages: E
  - Various kinds of languag
  - Procedural (C, C++, Pyt isp, Scheme, Haskell), Logic
- Execution: Combinatio ms
  - Hardware:
    - CPU (Transformations),
    - Memory (Primary and Second
    - Input/Output devices
  - Operating Systems and Systems Software: Processes and Files

Abstractions:

- Programming using language X

  Create useful abstraction of computing in design of language X.
  e.g. C : Memory viewed as (variable) names and types, Transformations are function names.

- Paradigms of Programming

  Some from the various different kinds of abstractions possible: Procedural, Object Oriented, Functional, ...

Key Theme

To build a computing system some of the things we need are:

- Algorithms: Capture th
  - Mathematical Models of
  - Complexity: How good t
- Languages: Express a
  - Various kinds of languag
  - Procedural (C, C++, Pyt isp, Scheme, Haskell), Logic
- Execution: Co ms
  - Hardware:
    - CPU (Transformations),
    - Memory (Primary and Secon
    - Input/Output devices
  - Operating Systems and Systems Software: Processes and Files

Abstractions:

- Computer Architecture

  Create Physical Devices for computing: Processing devices, Memory devices etc.

- Systems Software

  Create bridging software that realises abstractions from other areas in terms of created hardware.

- Operating Systems

  Create processes and files over given hardware.

Key Theme

To build a computing system some of the things we need are:

- Algorithms: Capture th
  - Mathematical models of
  - Complexity: How good t
- Languages: Express alg
  - Various kinds of language designs to relieve programming effort
  - Procedural (C, C++, Python, Java), OOP (C++, Java, Python), Functional (Lisp, Scheme, Haskell), Logic (Prolog), Strings based (Snobol), ...
- Execution: Combination of Hardware + Software to execute programs
  - Hardware:
    - CPU (Transformations),
    - Memory (Primary and Secondary) and
    - Input/Output devices
  - Operating Systems and Systems Software: Processes and Files

We will skip this part in our course.

Abstractions and Languages

The C Programming Language

Abstractions for Computing

- Data and Transformations (that manipulate data) within a program are represented using **names** for
  - Data – called variables, and
  - Transformations – called functions.
- Variables, i.e. data, have names and (data) types.
- Functions, i.e. transformations, have names and two components for transformations:
  - List of variables that supply information to transform, and
  - A variable to collect and return the information produced by the transfomations expressed in the function.

Abstractions and Languages

The C Programming Language

Abstractions for Computing

Programmer Needs

- Translate C code to binary
- Control translation details
- Organise code in multiple files
- Automate multi-file code compilation
- Manage multi-file development by multi-person teams

Programming Needs

- Two Ideas: Organise and Reuse
- Build large scale software
- Programming in teams
- Program versions in time

Abstractions and Languages

The C and (x86 or Y86) Programming Languages

Abstractions for Computing

C Language Abstractions

- Data: Numbers, Characters, ...
- Computing: Operators, Functions
- Control: Sequence, branch, ...
- Names: Use Words, scope, ...

x86 or Y86 ASM

- Data: bits, bytes, ...
- Computing: CPU Instructions set
- Control: (auto) Seq, JMP, ...
- Names: Numbers as Addresses

Binary Arithmetic: Place Value System

- Base 10 (aka Decimal)
  - $289 = 2 \times 10^2 + 8 \times 10^1 + 9 \times 10^0 = 200 + 80 + 9$
- Base 2: Binary
  - $100100001 = 1x2^8 + 0x2^7 + 0x2^6 + 1x2^5 + 0x2^4$

    $0x2^3 + 0x2^2 + 0x2^1 + 1x2^0$

    $= 256 + 0 + 0 + 32 + 0 + 0 + 0 + 1 = 289$
- Base 16: Hexadecimal
  - $121 = 1 \times 16^2 + 2 \times 16^1 + 1x\ 16^0 = 256 + 32 + 1$
- Base 8: Octal
  - $441 = 4 \times 8^2 + 4 \times 8^1 + 1 \times 8^0 = 256 + 32 + 1 = 289$

Binary Arithmetic: Place Value System

- Base 10 (aka Decimal)
  - 28
- Bas
  - 10
- Bas
  - 12
- Bas
  - 441 = 4 x 8^2 + 4 x 8^1 + 1 x 8^0 = 256 + 32 + 1 = 289

Follow our usual decimal methods for any base, but remember the base!

```
 1 1        1     Carry from previous digit addition!
   1100 0011
+  1110 1010
 ----------
  11010 1101
```

Y86 Architecture Summary (See text)

Byte	0	1	2	3	4	5
`halt`	0	0				
`nop`	1	0				
`rrmovl` **rA**, **rB**	2	0	**rA**	**rB**		
`irmovl` **V**, **rB**	3	0	F	**rB**	**V**	
`rmmovl` **rA**, **D**(**rB**)	4	0	**rA**	**rB**	**D**	
`mrmovl` **D**(**rB**), **rA**	5	0	**rA**	**rB**	**D**	
`OPl` **rA**, **rB**	6	**fn**	**rA**	**rB**		
`jXX` **Dest**	7	**fn**	**Dest**			
`cmovXX` **rA**, **rB**	2	**fn**	**rA**	**rB**		
`call` **Dest**	8	0	**Dest**			
`ret`	9	0				
`pushl` **rA**	A	0	**rA**	F		
`popl` **rA**	B	0	**rA**	F		

RF: Program registers

%eax	%esi
%ecx	%edi
%edx	%esp
%ebx	%ebp

CC: Condition codes

ZF	SF	OF

PC

Stat: Program status

DMEM: Memory

Operations

`addl`	6	0
`subl`	6	1
`andl`	6	2
`xorl`	6	3

Branches

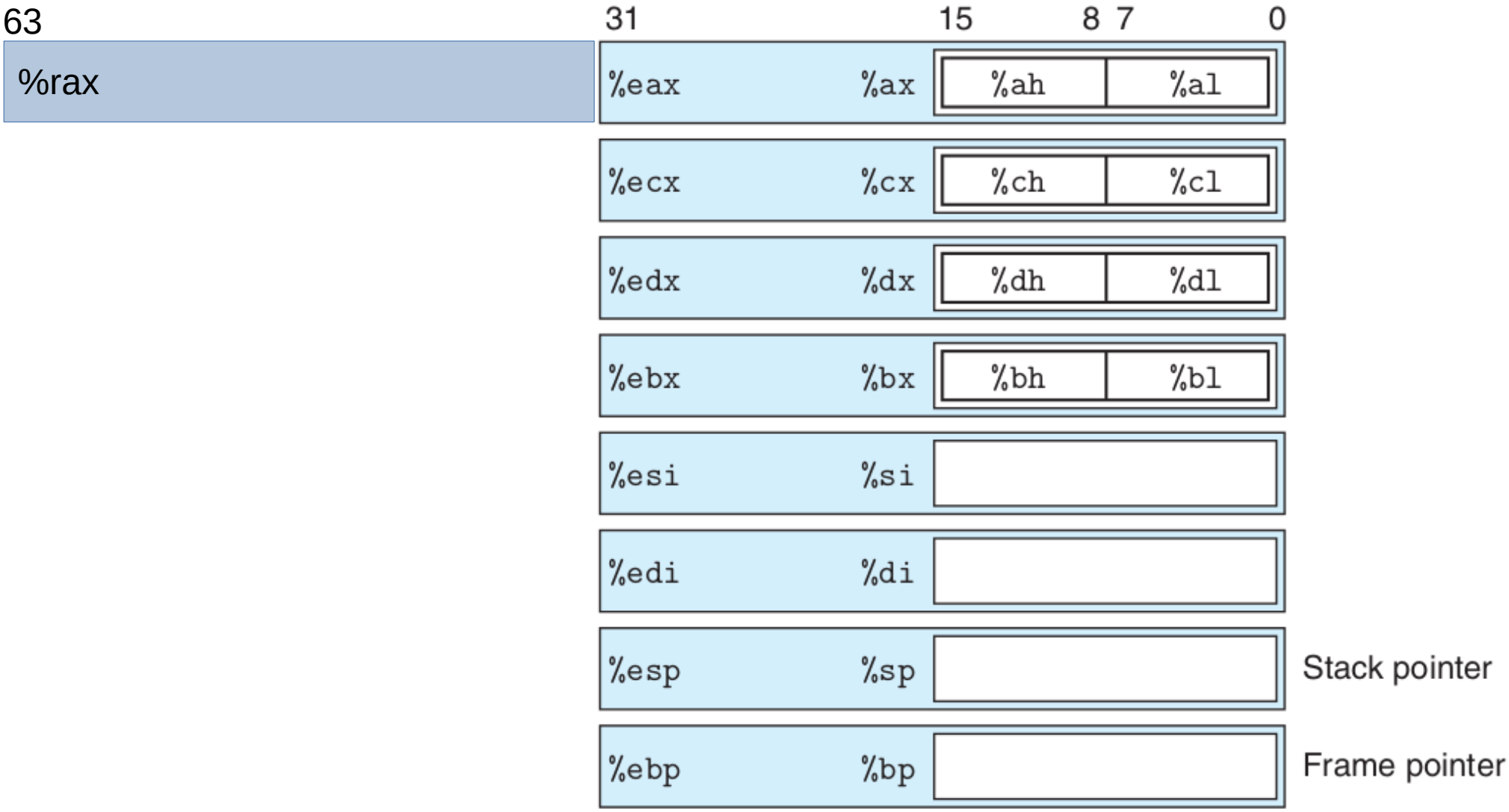
`jmp`	7	0
`jle`	7	1
`jl`	7	2
`je`	7	3
`jne`	7	4
`jge`	7	5
`jg`	7	6

Moves

`rrmovl`	2	0
`cmovle`	2	1
`cmovl`	2	2
`cmove`	2	3
`cmovne`	2	4
`cmovge`	2	5
`cmovg`	2	6

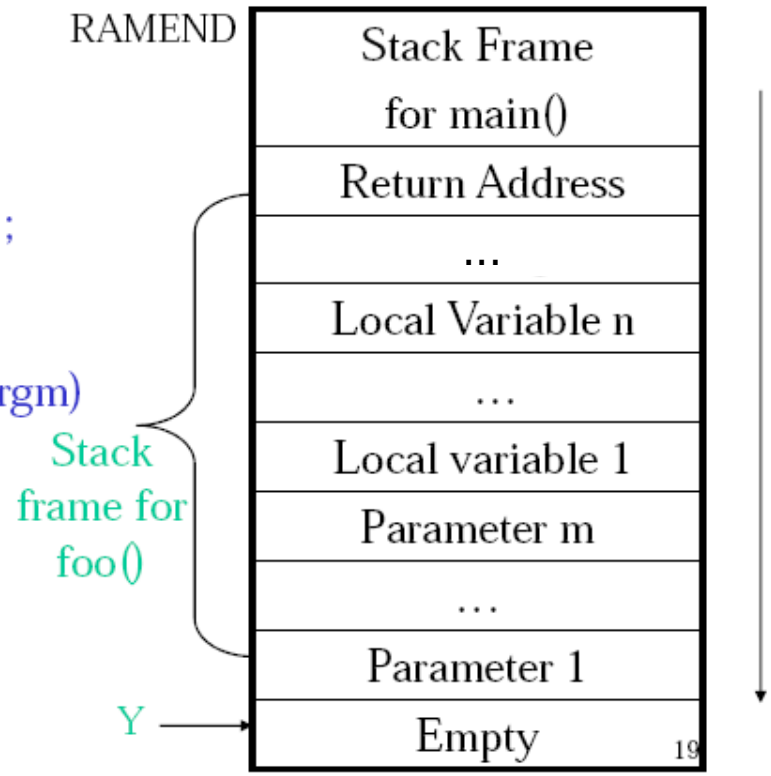
Number	Register name
0	%eax
1	%ecx
2	%edx
3	%ebx
4	%esp
5	%ebp
6	%esi
7	%edi
F	No register

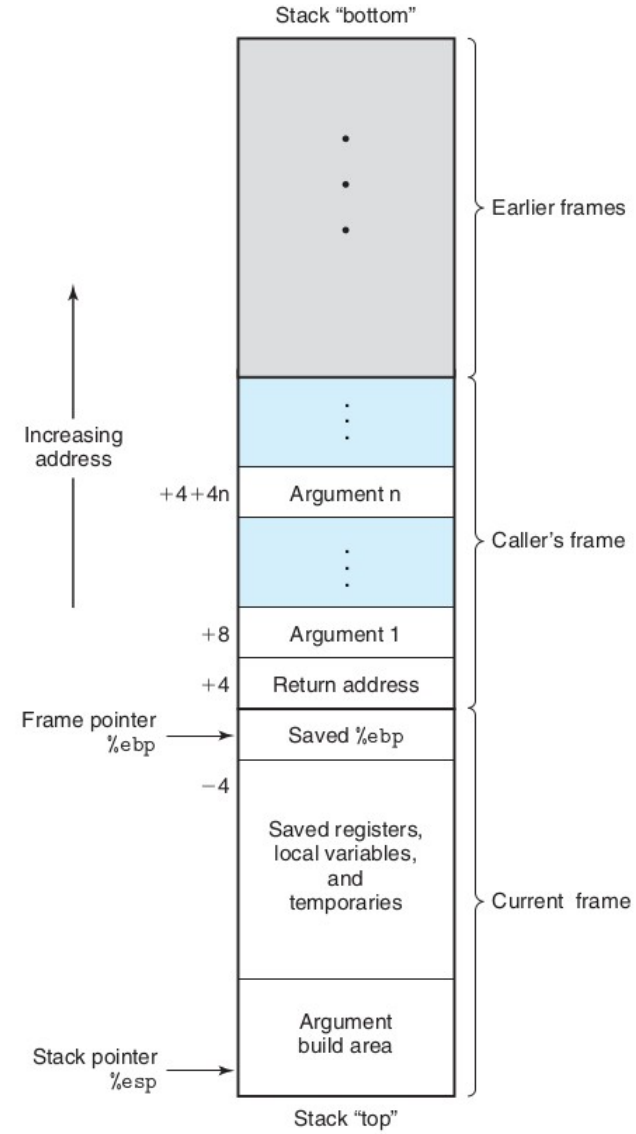
x86 Registers



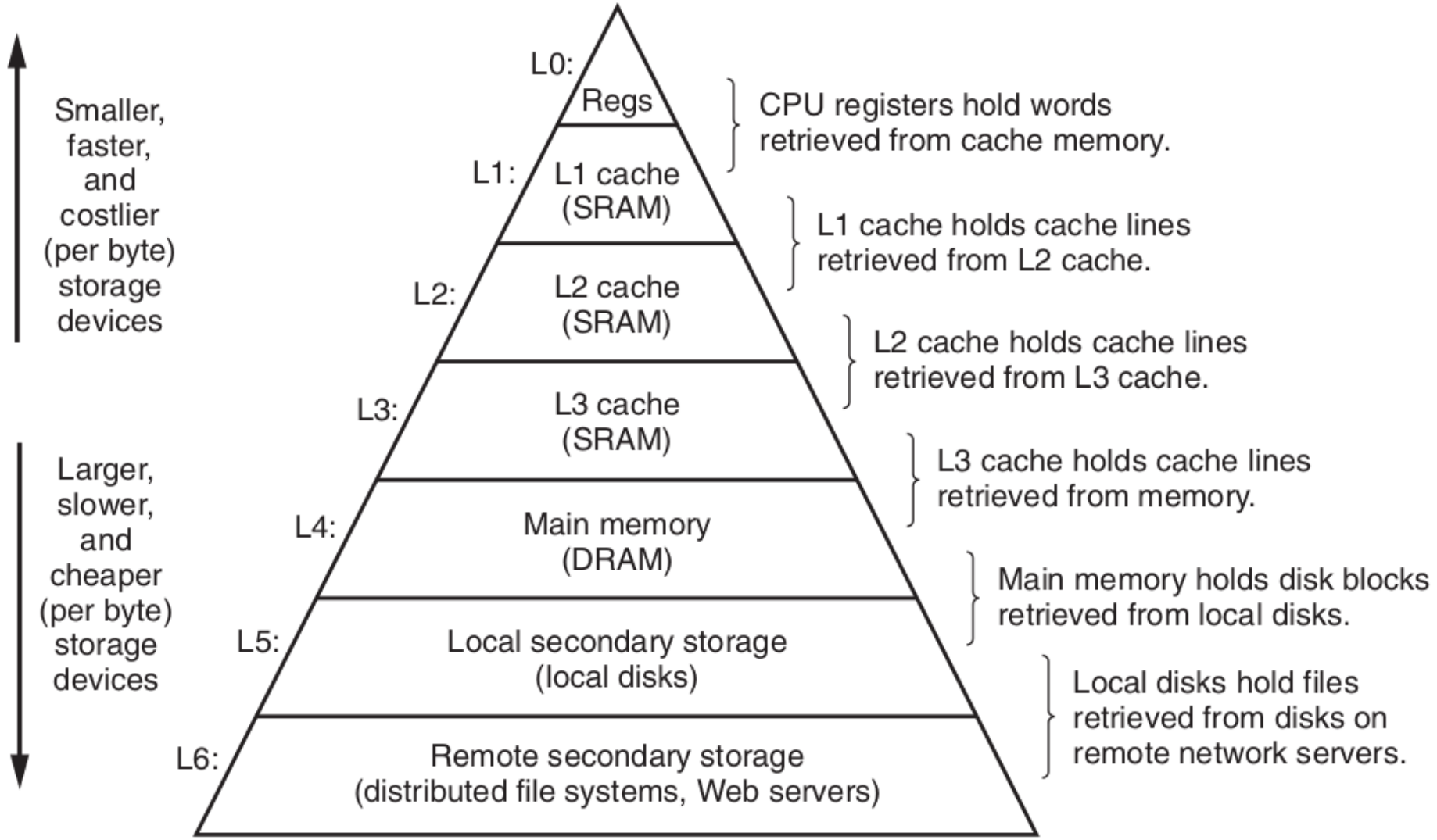
The Activation Record: Stack of Activations

```
int main(void)
{ …
  foo(arg1, arg2, …, argm);
}
void foo(arg1, arg2, …, argm)
{ int var1, var2, …, varn;
  …
}
```

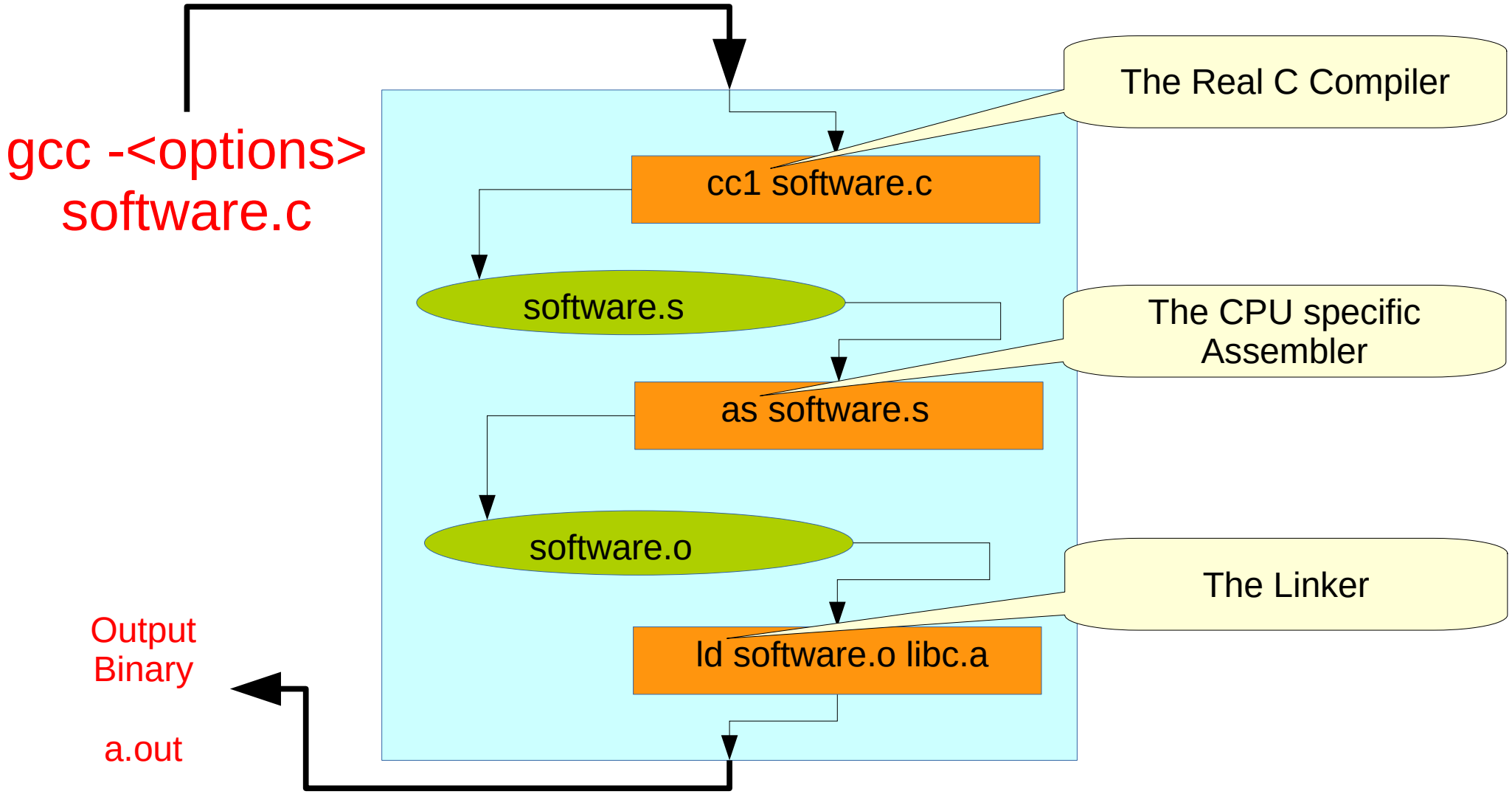




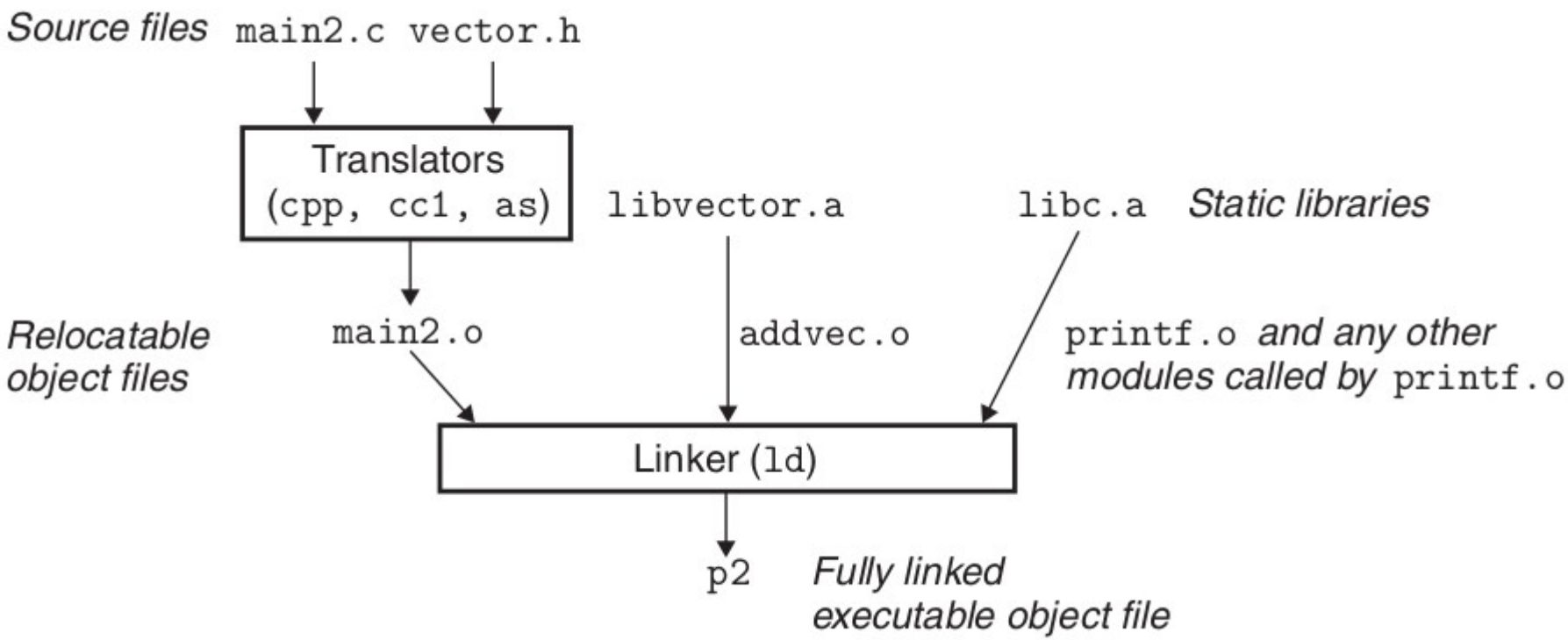
Memory Hierarchy



The “gcc” Compiler Driver



Static Linking



(Simple) Relocation Algorithm

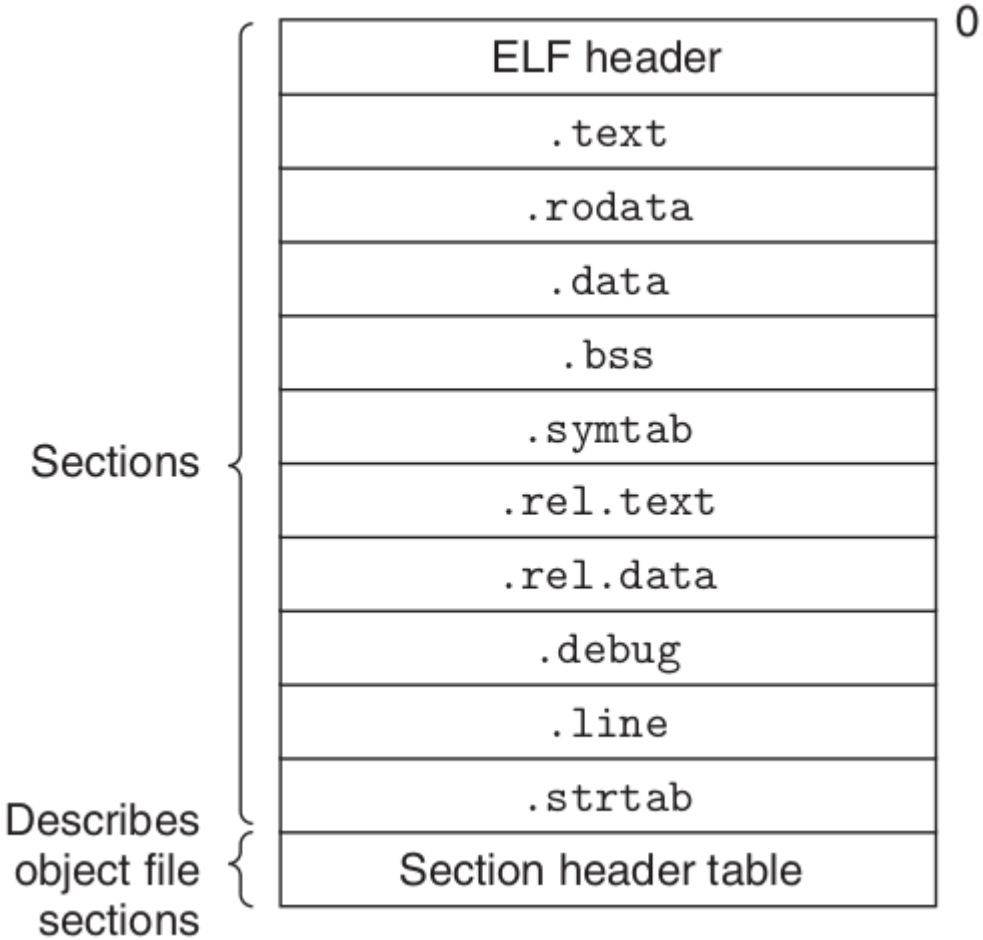
(See ELF Details)

```
foreach section s {
    foreach relocation entry r {
        refptr = s + r.offset;  /* ptr to reference to be relocated */

        /* Relocate a PC-relative reference */
        if (r.type == R_386_PC32) {
            refaddr = ADDR(s) + r.offset; /* ref's run-time address */
            *refptr = (unsigned) (ADDR(r.symbol) + *refptr - refaddr);
        }

        /* Relocate an absolute reference */
        if (r.type == R_386_32)
            *refptr = (unsigned) (ADDR(r.symbol) + *refptr);
    }
}
```

(Sketch of) The ELF Format



`.data`: *Initialized* global C variables. Local C variables are maintained at run time on the stack, and do `not` appear in either the `.data` or `.bss` sections.

`.bss`: *Uninitialized* global C variables. This section occupies no actual space in the object file; it is merely a place holder. Object file formats distinguish between initialized and uninitialized variables for space efficiency: uninitialized variables do not have to occupy any actual disk space in the object file.

`.symtab`: A *symbol table* with information about functions and global variables that are defined and referenced in the program. Some programmers mistakenly believe that a program must be compiled with the `-g` option to get symbol table information. In fact, every relocatable object file has a symbol table in `.symtab`. However, unlike the symbol table inside a compiler, the `.symtab` symbol table does not contain entries for local variables.

`.rel.text`: A list of locations in the `.text` section that will need to be modified when the linker combines this object file with others. In general, any instruction that calls an external function or references a global variable will need to be modified. On the other hand, instructions that call local functions do not need to be modified. Note that relocation information is not needed in executable object files, and is usually omitted unless the user explicitly instructs the linker to include it.

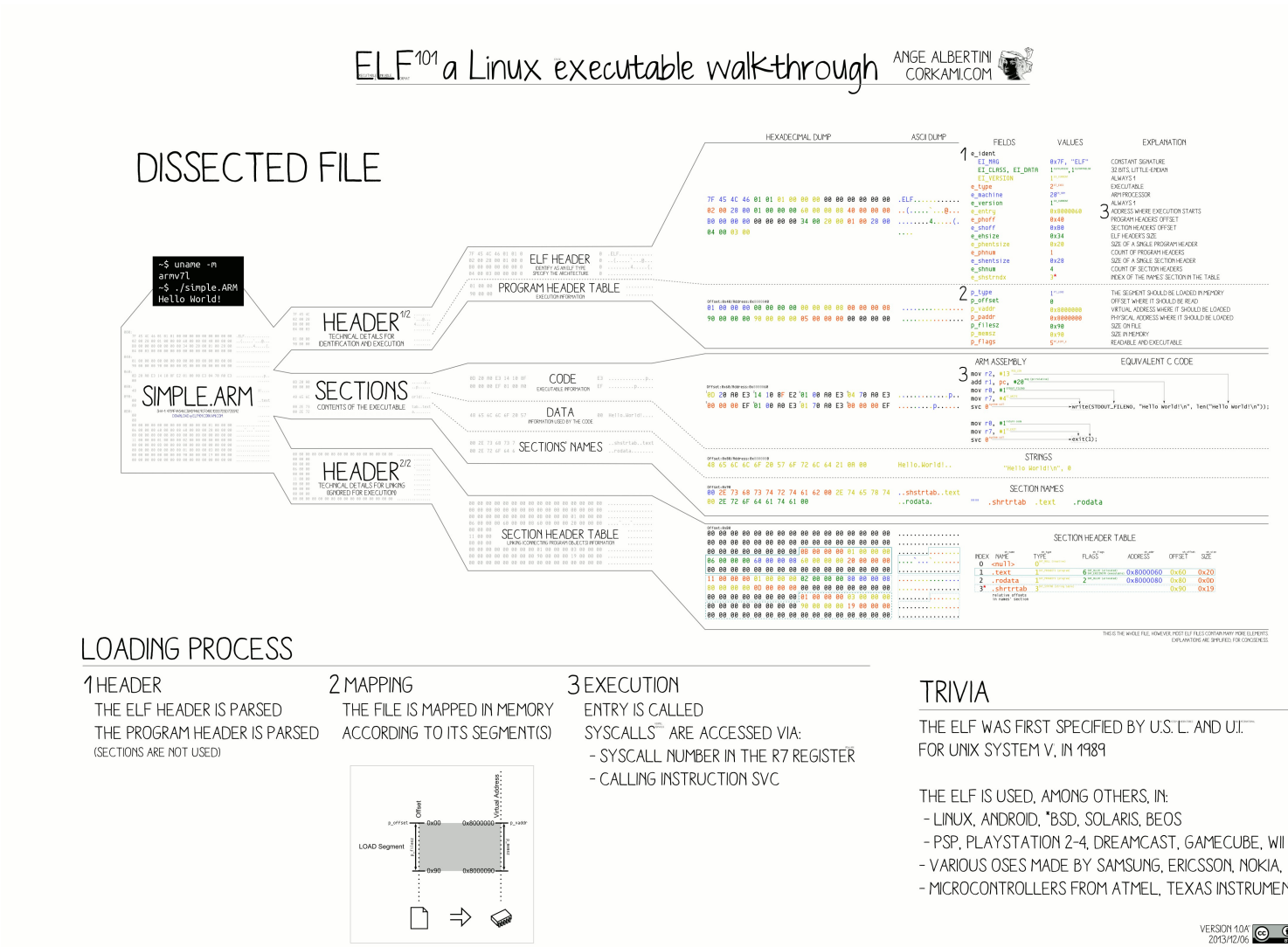
`.rel.data`: Relocation information for any global variables that are referenced or defined by the module. In general, any initialized global variable whose initial value is the address of a global variable or externally defined function will need to be modified.

`.debug`: A debugging symbol table with entries for local variables and typedefs defined in the program, global variables defined and referenced in the program, and the original C source file. It is only present if the compiler driver is invoked with the `-g` option.

`.line`: A mapping between line numbers in the original C source program and machine code instructions in the .text section. It is only present if the compiler driver is invoked with the `-g` option.

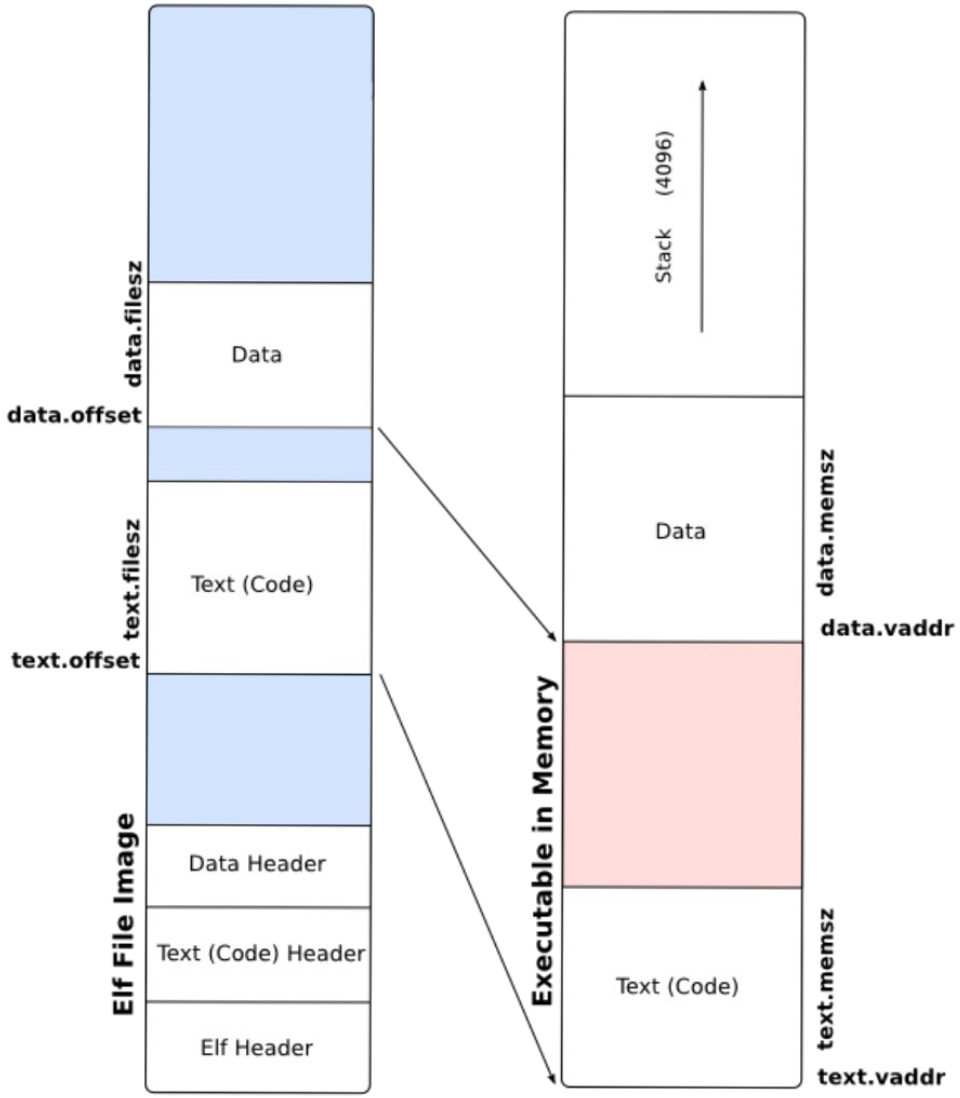
`.strtab`: A string table for the symbol tables in the `.symtab` and `.debug` sections, and for the section names in the section headers. A string table is a sequence of null-terminated character strings.

(Sketch of) The ELF Format



Source

Program ► Process



Source: OS Dev

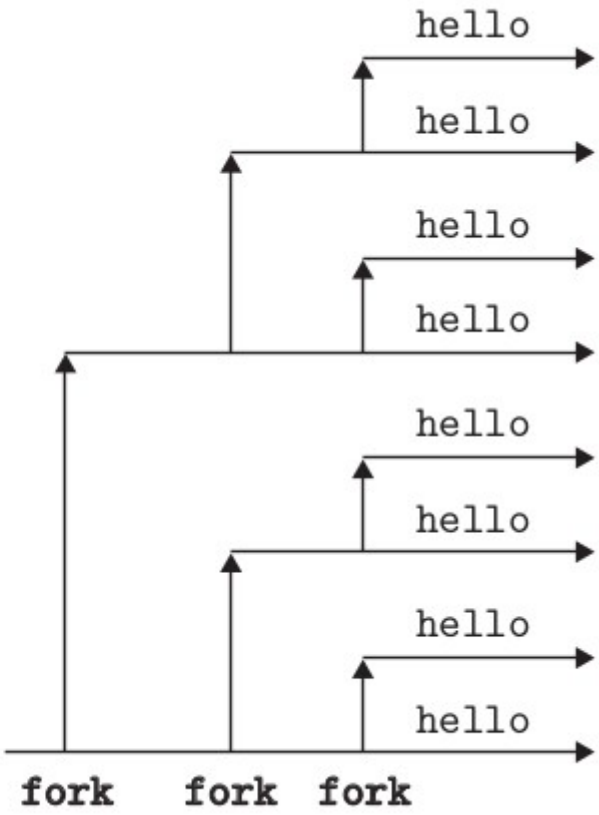
Effect of fork()

Program

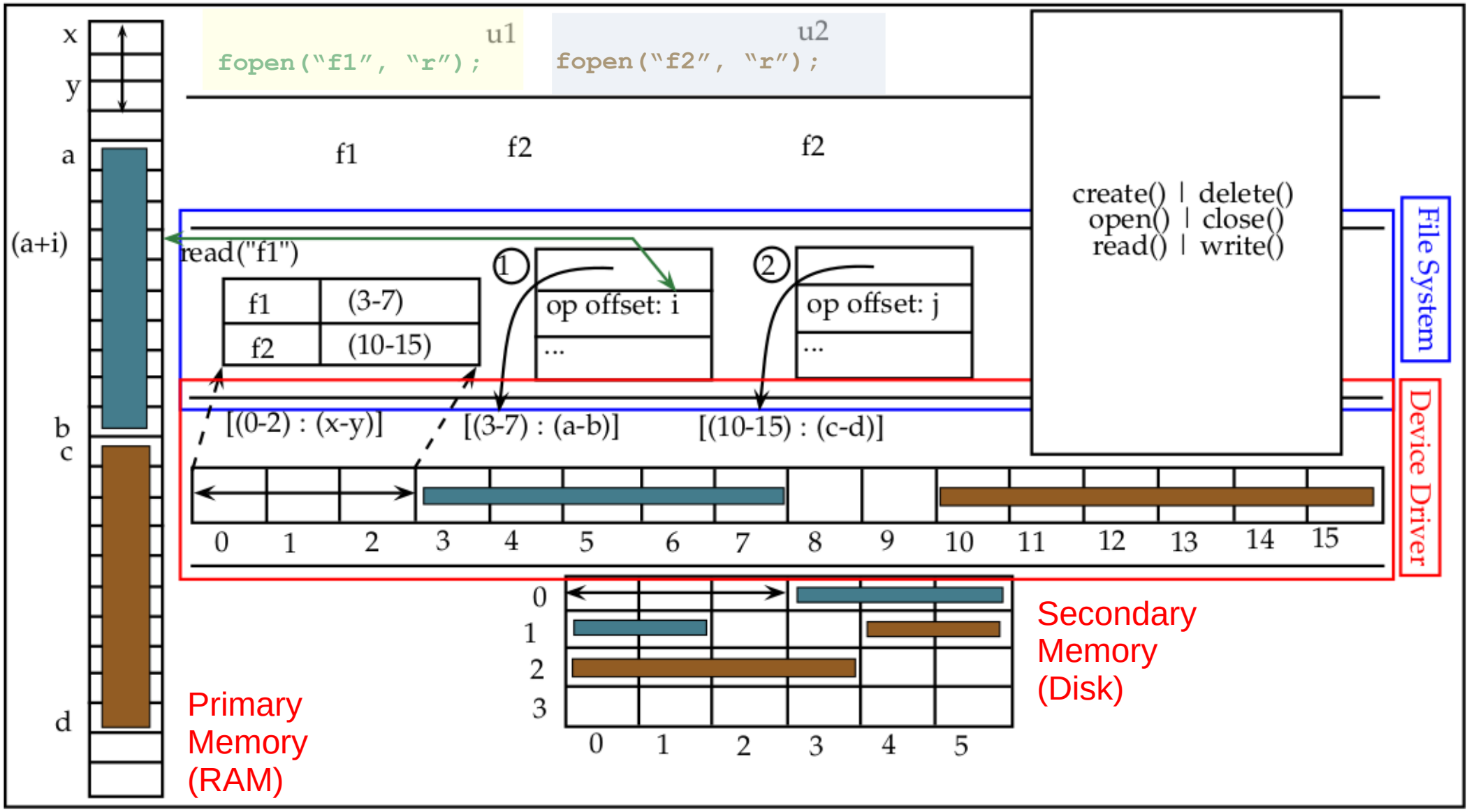
```
#include "csapp.h"

int main()
{
    Fork();
    Fork();
    Fork();
    printf("hello\n");
    exit(0);
}
```

Process



The “File” Abstraction



Thank You & Questions

To Be Continued ...